



Universidad Mariano Galvez de Guatemala
Facultad de Ingeniería, Matemáticas y Ciencias Físicas
Ingeniería en Sistemas
Programación III
Catedrático: Ing. David Alvarez

Tarea XIII BFS y DFS

Jeremy Javier Lima Guitron
9941-22-9790
31/05/2026

Parte 1 – Recorridos manuales de grafos

Instrucciones

Para cada ejercicio, el grupo debe:

- Dibujar el grafo.
- Escribir la lista de adyacencia.
- Realizar el recorrido BFS desde el nodo indicado.
- Realizar el recorrido DFS desde el nodo indicado.
- Indicar el orden de visita.

Cuando exista más de un vecino disponible, deben visitar los nodos en orden ascendente o alfabético.

	Ejercicio 2	
Ejercicio 1 1-2 1-3 2-4 3-5 Inicio: 1 Grafo: <pre> 1 / \ 2 3 / \ 4 5 </pre> BFS: 1-2-3-4-5 DFS: 1-2-4-3-5 Adj: 1 2,3 2 1,4 3 1,5 4 2 5 3	Ejercicio 2 1-2 1-4 2-3 4-5 5-6 Inicio: 1 Grafo: <pre> 1 / \ 2 4 / \ 3 5 \ 6 </pre> BFS: 1-2-4-3-5-6 DFS: 1-2-3-4-5-6 Adj: 1 2,4 2 1,3 3 2 4 1,5 5 4,6 6 5	Ejercicio 3 A-B A-C B-D B-E C-F Inicio: A Grafo: <pre> A / \ B C / \ \ D E F </pre> BFS: A-B-C-D-E-F DFS: A-B-D-E-C-F Adj: A B,C B A,D,E C A,F D B E B F C

Ejercicio 4 1-2 2-3 3-4 4-5 Inicio: 1 Grafo 1 - 2 - 3 - 4 - 5 BFS: 1-2-3-4-5 DFS: 1-2-3-4-5 Adj: 1 2 2 1,3 3 2,4 4 3,5 5 4	Ejercicio 5 1-2 1-3 1-4 2-5 4-6 Inicio: 1 Grafo <pre> 1 / \ 2 3 4 / \ 5 6 </pre> BFS: 1-2-3-4-5-6 DFS: 1-2-5-3-4-6 Adj: 1 2,3,4 2 1,5 3 1 4 1,6 5 2 6 4	Ejercicio 6 A-B B-C C-D A-E E-F Inicio: A Grafo A - B - C - D E - F BFS: A-B-E-C-F-D DFS: A-B-C-D-E-F Adj: A B,E B A,C C B,D D C E A,F F E
Ejercicio 7 1-2 1-3 2-4 2-5 3-6 3-7 Inicio: 1 Grafo <pre> 1 / \ 2 3 </pre>	Ejercicio 8 A-B A-C B-D C-D D-E Inicio: A Grafo <pre> A \ \ B C \ / </pre>	Ejercicio 9 1-2 2-3 3-1 3-4 4-5 Inicio: 1 Grafo <pre> 1 / \ 2---3---4---5 </pre>

<pre> / \ / \ 4 5 6 7 </pre> BFS: 1-2-3-4-5-6-7 DFS: 1-2-4-5-3-6-7 Adj: <pre> 1 2,3 2 1,4,5 3 1,6,7 4 2 5 2 6 3 7 3 </pre>	<pre> D E </pre> BFS: A-B-C-D-E DFS: A-B-D-C-E Adj: <pre> A B,C B A,D C A,D D B,C,E E D </pre>	BFS: 1-2-3-4-5 DFS: 1-2-3-4-5 Adj: <pre> 1 2,3 2 1,3 3 1,2,4 4 3,5 5 4 </pre>
Ejercicio 10 A-B A-D B-C D-E C-F E-F Inicio: A Grafo <pre> A - B - C D - E - F </pre> BFS: A-B-D-C-E-F DFS: A-B-C-F-E-D Adj: <pre> A B,D B A,C C B,F D A,E E D,F F C,E </pre>	Ejercicio 11 1-2 1-5 2-3 2-4 5-6 6-7 Inicio: 1 Grafo <pre> 1 / \ 2 5 / \ \ 3 4 6 \ 7 </pre> BFS: 1-2-5-3-4-6-7 DFS: 1-2-3-4-5-6-7 Adj: <pre> 1 2,5 2 1,3,4 3 2 4 2 5 1,6 </pre>	Ejercicio 12 A-B A-C B-E C-D D-E E-F Inicio: A Grafo <pre> A \ \ B C E--D F </pre> BFS: A-B-C-E-D-F DFS: A-B-E-D-C-F Adj: <pre> A B,C B A,E C A,D D C,E </pre>

	6 5,7 7 6	E B,D,F F E
Ejercicio 13 1-2 1-3 2-4 4-6 3-5 5-6 6-7 Inicio: 1 Grafo <pre> 1 / \ 2 3 4 5 \ / 6 7 </pre> BFS: 1-2-3-4-5-6-7 DFS: 1-2-4-6-5-3-7 Adj: <pre> 1 2,3 2 1,4 3 1,5 4 2,6 5 3,6 6 4,5,7 7 6 </pre>	Ejercicio 14 A-B B-C C-D D-A A-E E-F Inicio: A Grafo <pre> A---B D---C E---F </pre> BFS: A-B-D-E-C-F DFS: A-B-C-D-E-F Adj: <pre> A B,D,E B A,C C B,D D A,C E A,F F E </pre>	Ejercicio 15 1-2 1-3 2-5 3-4 4-6 5-6 6-7 7-8 Inicio: 1 Grafo <pre> 1 / \ 2 3 5 4 \ / 6 7 8 </pre> BFS: 1-2-3-5-4-6-7-8 DFS: 1-2-5-6-4-3-7-8 Adj: <pre> 1 2,3 2 1,5 3 1,4 4 3,6 5 2,6 6 4,5,7 7 6,8 8 7 </pre>

Ejercicio 16

1-2

1-3

2-4

2-5

3-6

5-7

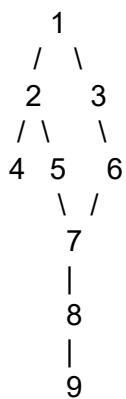
6-7

7-8

8-9

Inicio: 1

Grafo



BFS: 1-2-3-4-5-6-7-8-9

DFS: 1-2-4-5-7-6-3-8-9

Adj:

1 | 2,3

2 | 1,4,5

3 | 1,6

4 | 2

5 | 2,7

6 | 3,7

7 | 5,6,8

8 | 7,9

9 | 8

Responda:

¿En qué momento (nivel) BFS llega al nodo 9?

BFS llega al nodo 9 en el nivel 5

¿DFS visita el nodo 9 antes o después que BFS?

DFS visita el nodo 9 al mismo tiempo que BFS porque ambos lo encuentran al final del recorrido

¿Cuál recorrido considera más eficiente en este caso y por qué?

Para este grafo, BFS es ligeramente más conveniente, porque muestra que el nodo 9 se encuentra a 5 niveles del nodo inicial y garantiza encontrar el camino más corto

Parte 2 – Comparación técnica C++ vs Java

El grupo deberá ejecutar dos implementaciones funcionales del mismo grafo:

1. Una versión en C++ puro, sin usar `vector`, `queue`, `stack`, `map` ni `unordered_map`.
2. Una versión en Java, usando `HashMap`, `ArrayList`, `Queue` y `LinkedList`.

Luego deberán elaborar una comparación técnica.

[Click aquí para ir al código en C++](#)

[Click aquí para ir al código en Java](#)

Análisis técnico requerido

El grupo debe responder:

1. ¿Qué estructuras se implementaron manualmente en C++?

En C++ se implementaron manualmente varias estructuras de datos porque no se permitió utilizar bibliotecas como *vector*, *queue* o *map*.

Las estructuras creadas manualmente fueron:

- **Lista de adyacencia:** mediante la estructura *NodoAdyacente*.
- **Cola (Queue):** mediante la estructura *NodoCola* y la clase *Cola*.
- **Arreglo dinámico de vértices:** utilizando *new Vertice*.
- **Arreglo dinámico de visitados:** utilizando *new bool*.

Esto obliga a gestionar directamente la memoria y los enlaces entre nodos.

2. ¿Qué estructuras ya existen en Java?

Java nos da estructuras listas para usar mediante sus bibliotecas estándar.

En este programa se utilizaron:

- ***HashMap<Integer, List<Integer>>*** para representar el grafo.
- ***ArrayList<Integer>*** para almacenar vecinos.
- ***HashSet<Integer>*** para registrar nodos visitados.
- ***Queue<Integer>*** para la cola de BFS.
- ***LinkedList<Integer>*** como implementación de la cola.

Estas estructuras reducen considerablemente la cantidad de código necesario.

3. ¿Cuál código es más largo?

El código de C++ es más largo.

Razones:

- Debe implementar la cola manualmente.
- Debe crear estructuras para nodos y listas enlazadas.
- Debe reservar memoria con *new*.
- Debe liberar memoria con *delete*.
- Necesita un destructor para evitar fugas de memoria.

Java utiliza clases ya implementadas, por lo que requiere menos líneas de código.

4. ¿Cuál código es más fácil de entender?

El código de Java es más fácil de entender porque:

- Utiliza nombres de estructuras conocidas (*HashMap*, *ArrayList*, *Queue*).
- No muestra detalles internos de memoria.
- La sintaxis es más limpia.
- El recorrido BFS y DFS se observa de forma más directa.

En C++ parte del código se dedica a administrar memoria y punteros.

5. ¿Cuál lenguaje exige mayor control del programador?

C++ exige mayor control del programador ya que debe:

- Reservar memoria con *new*.
- Liberar memoria con *delete*.
- Manejar punteros.
- Evitar accesos inválidos.
- Implementar estructuras de datos manualmente.

6. ¿Qué ventajas tiene C++ en este caso?

- Mayor control sobre la memoria.
- Mayor eficiencia y rendimiento.
- Permite comprender cómo funcionan internamente las estructuras de datos.
- Menor dependencia de bibliotecas externas.

7. ¿Qué ventajas tiene Java en este caso?

- Menor cantidad de código.
- Desarrollo más rápido.
- Mayor legibilidad.
- Menor riesgo de errores de memoria.
- Amplia biblioteca de estructuras de datos ya implementadas.

8. ¿Qué sucede si en C++ no se libera memoria correctamente?

Si la memoria reservada con `new` no se libera mediante `delete`, se producen fugas de memoria (*Memory Leaks*).

Consecuencias:

- El programa consume cada vez más memoria.
- Puede disminuir el rendimiento.
- Puede provocar fallos del sistema.
- En aplicaciones grandes puede causar cierres inesperados.

9. ¿Por qué BFS necesita una cola?

BFS explora los nodos por niveles.

La cola permite que:

1. El primer nodo que entra sea el primero en salir (FIFO).
2. Se visiten primero los vecinos más cercanos.
3. Se mantenga el orden correcto de exploración.

10. ¿Por qué DFS puede resolverse con recursión?

DFS avanza por un camino hasta el final antes de retroceder. La recursión funciona porque cada llamada se guarda automáticamente en la pila de ejecución del sistema.

Proceso:

- Visita un nodo.
- Llama recursivamente a un vecino.
- Continúa profundizando.
- Cuando no hay más vecinos, regresa al nodo anterior.



Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas

Nombre:	Javier Lima	Carné:	9941-22-9790
Nombre:	—	Carné:	—
Nombre:	—	Carné:	—

Carrera	Ingeniería en Sistemas	Carrera	9941
Asignatura	Programación III	Curso	022
Ciclo	Quinto Ciclo	Jornada	Domingos
Catedrático	Ing. MBA David Alvarez	Fecha	24/05/2026
Semestre	Primer Semestre	Sección	B

Objetivo

Aplicar recorridos de grafos mediante BFS y DFS, representando gráficamente los grafos, resolviendo recorridos manualmente y comparando una implementación en C++ puro contra una implementación en Java con Collections Framework.

Parte 1 – Recorridos manuales de grafos

Instrucciones

Para cada ejercicio, el grupo debe:

1. Dibujar el grafo.
2. Escribir la lista de adyacencia.
3. Realizar el recorrido BFS desde el nodo indicado.
4. Realizar el recorrido DFS desde el nodo indicado.
5. Indicar el orden de visita.

Cuando exista más de un vecino disponible, deben visitar los nodos en orden ascendente o alfabético.

Ejercicio 1 1-2 1-3 2-4 3-5 Inicio: 1	Ejercicio 2 1-2 1-4 2-3 4-5 5-6 Inicio: 1	Ejercicio 3 A-B A-C B-D B-E C-F Inicio: A
Ejercicio 4 1-2 2-3 3-4 4-5 Inicio: 1	Ejercicio 5 1-2 1-3 1-4 2-5 4-6 Inicio: 1	Ejercicio 6 A-B B-C C-D A-E E-F Inicio: A
Ejercicio 7 1-2 1-3 2-4 2-5 3-6 3-7 Inicio: 1	Ejercicio 8 A-B A-C B-D C-D D-E Inicio: A	Ejercicio 9 1-2 2-3 3-1 3-4 4-5 Inicio: 1
Ejercicio 10 A-B A-D B-C D-E C-F E-F Inicio: A	Ejercicio 11 1-2 1-5 2-3 2-4 5-6 6-7 Inicio: 1	Ejercicio 12 A-B A-C B-E C-D D-E E-F Inicio: A
Ejercicio 13 1-2 1-3 2-4 4-6 3-5 5-6 6-7 Inicio: 1	Ejercicio 14 A-B B-C C-D D-A A-E E-F Inicio: A	Ejercicio 15 1-2 1-3 2-5 3-4 4-6 5-6 6-7 7-8 Inicio: 1
Ejercicio 16 1-2 1-3 2-4 2-5 3-6 5-7 6-7 7-8 8-9 Inicio: 1	Ejercicio 17 Dibuje el grafo. Escriba la lista de adyacencia. Realice el recorrido BFS desde el nodo 1. Realice el recorrido DFS desde el nodo 1. Responda: • ¿En qué momento (nivel) BFS llega al nodo 9? • ¿DFS visita el nodo 9 antes o después que BFS? • ¿Cuál recorrido considera más eficiente en este caso y por qué?	

Recorridos Manuales de Grafos

① 1-2 1 • BFS: 1-2-3-4-5
 1-3 1 \ • DFS: 1-2-4-3-5
 2-4 2 3 • Adj: 1|2,3
 3-5 1 1 2|4
 Inicio: 1 4 5 3|5

② 1-2 1 • BFS: 1-2-4-3-5-6
 1-4 1 1 • DFS: 1-2-3-4-5-6
 2-3 2 4 • Adj: 1|2,4
 4-5 1 1 2|3
 5-6 3 5 4|5
 Inicio: 1 1 5|6
 6

③ A-B A • BFS: A-B-C-D-E-F
 A-C 1 1 • DFS: A-B-D-E-C-F
 B-D B C • Adj: A|B,C
 B-E 1 1 1 B|D
 C-F D E F D|E
 Inicio: A C|F

④ 1-3 1 • BFS: 1-2-3-4-5
 2-4 1 • DFS: 1-2-3-4-5
 3-4 3 2 • Adj: 1|2
 3-5 1 2|3
 Inicio: 1 3 3|4
 1 4|5
 4
 1
 5

